CARGO PUSH

안중현
Portfolio

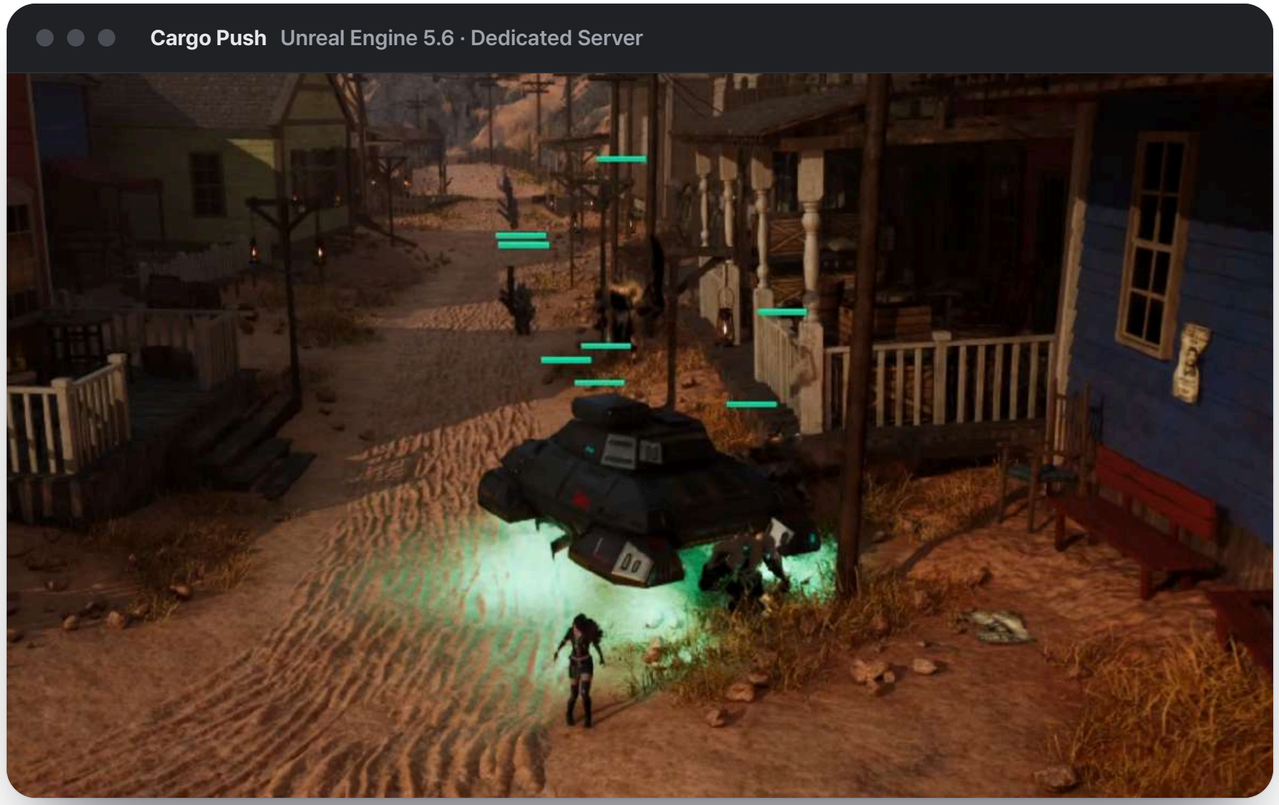
PERSONAL PROJECT

언리얼 엔진 5 팀 대전 멀티플레이 화물 밀기 게임

데디케이티드 서버 기반 팀 대전 게임
GAS 스킬 전투, EOS 로그인, Python 코디네이터 서버 구현

GAS | EOS | Dedicated Server | Asset Manager

Unreal Engine 5.6 · C++ / Blueprint



안중현 · sie08357@naver.com · 010-2070-2986

OVERVIEW

프로젝트 개요

데디케이티드 서버 기반, 화물 밀기 방식의 팀 멀티플레이 게임



GitHub ↗ 플레이 영상 ↗ 블로그 ↗

기본 정보

엔진	Unreal Engine 5.6
툴	C++, Blueprint 혼합
인원	개인 개발
아키텍처	GAS, EOS, Asset Manager
네트워크	RPC, Handle System, Replication Dedicated Server, HTTP

목표 및 컨셉

01 화물 밀기 팀 대전

데디케이티드 서버 기반 팀 멀티플레이

02 GAS 스킬 전투

Gameplay Ability System 기반 스킬 전투 시스템

03 EOS 로그인 · HTTP 서버

EOS 로그인 연동, Python HTTP 서버로 서버-클라이언트 통신

INDEX

목차

Frontend · 로그인 및 로비 · 캐릭터 · 인벤토리 · AI · 화물



01 Frontend

Common UI 기반 메뉴 · 옵션 플러그인



02 로그인 및 로비

EOS 로그인, HTTP 세션 생성, 캐릭터 선택



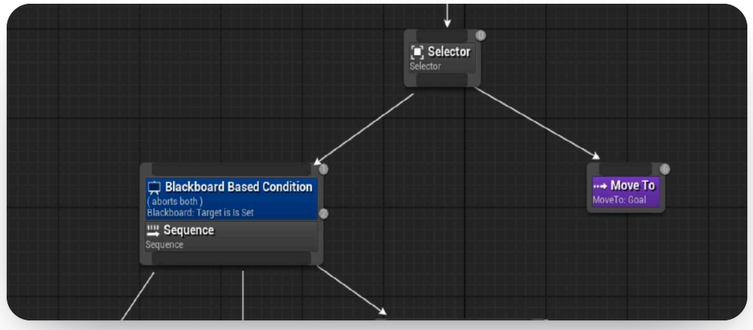
03 캐릭터

GAS 스탯 · 어빌리티, 스윕 충돌 판정



04 인벤토리

서버 권한 구조, 핸들 기반 동기화



05 AI

오브젝트 풀링 스폰, Behavior Tree



06 화물

점령 인원 차이에 따른 목표 이동

01 · FRONTEND

Common UI 프론트엔드 플러그인

게임 시작 시 표시되는 메인 화면. 로그인, 설정, 크레딧, 종료 기능 제공

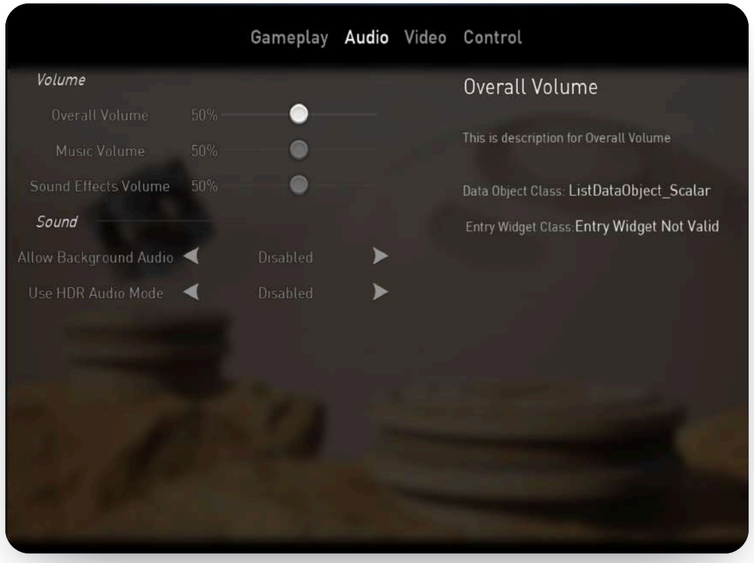
메인 화면

로그인 · 옵션 · 크레딧 · 종료 메뉴



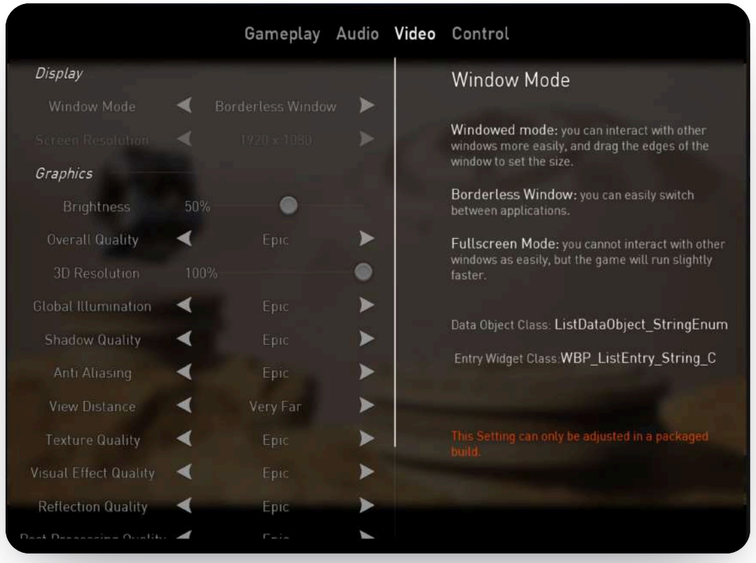
오디오 설정

전체 · 음악 · 효과음 볼륨, 사운드 옵션



비디오 설정

창 모드, 그래픽 품질 옵션과 항목별 설명



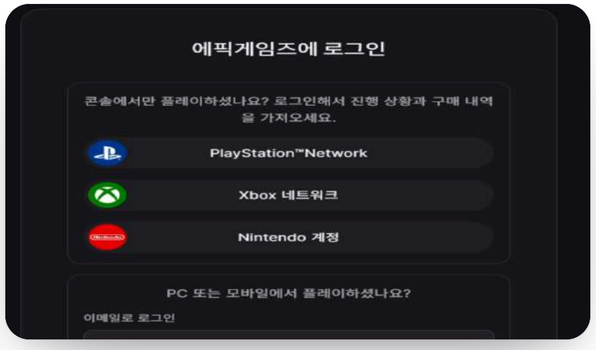
플러그인 구조 메뉴, 시작 화면, 옵션을 일관된 구조로 구성. 다른 프로젝트에도 그대로 적용 가능

CommonFront GitHub ↗

EOS 로그인

로그인 버튼 클릭 시 EOS(Epic Online Services)로 로그인 요청 전송





UCGameInstance::ClientLogin

```cpp
void UCGameInstance::ClientLogin(const FString& Type, const FString& Id, const FString& Token)
{
    if (IOnlineIdentityPtr IdentityPtr = UCNetStatics::GetIdentityPtr())
    {
        if (LoggingInDelegateHandle.IsValid())
        {
            IdentityPtr->OnLoginCompleteDelegates->Remove(LoggingInDelegateHandle);
            LoggingInDelegateHandle.Reset();
        }

        LoggingInDelegateHandle = IdentityPtr->OnLoginCompleteDelegates->AddUObject(this, &UCGameInstance::LoginCompleted);
        if (!IdentityPtr->Login( LocalUserNum: 0, FOnlineAccountCredentials(Type, Id, Token)))
        {
            UE_LOG(LogTemp, Warning, TEXT("Login Failed Right Away!"))
            if (LoggingInDelegateHandle.IsValid())
            {
                IdentityPtr->OnLoginCompleteDelegates->Remove(LoggingInDelegateHandle);
                LoggingInDelegateHandle.Reset();
            }
            OnLoginCompleted.Broadcast(false, "", "Login Failed Right Away!");
        }
    }
}
```

01 로그인 요청 수신

로그인 버튼 클릭 시 클라이언트에서 로그인 정보 수집

02 EOS 로그인 시도

계정 정보(Type, ID, Token)를 EOS에 전달해 로그인 요청

03 로그인 완료 처리

성공 · 실패 결과를 콜백으로 받아 후속 작업 수행

04 즉시 실패 처리

요청이 바로 실패한 경우 UI와 시스템에 실패 전달

처리 흐름

클라이언트 → EOS → 콜백 처리 → UI / 시스템 알림

HTTP 로비 생성

언리얼 HTTP 모듈로 Python 코디네이터 서버에 세션 생성 요청

UCGameInstance::RequestCreateAndJoinSession

```cpp
void UCGameInstance::RequestCreateAndJoinSession(const FName& NewSessionName)
{
    UE_LOG(LogTemp, Warning, TEXT("Requesting Create and Join Session: %s"), *(NewSessionName.ToString()))
    FHttpRequestRef Request = FHttpModule::Get().CreateRequest();
    FGuid SessioinSearchId = FGuid::NewGuid();

    FString CoordinatorURL = UCNetStatics::GetCoordinatorURL();

    FString URL = FString::Printf( Fmt TEXT("%s/Sessions"), *CoordinatorURL);
    UE_LOG(LogTemp, Warning, TEXT("Sending Request Session Creation to URL: %s"), *URL)

    Request->SetURL(URL);
    Request->SetVerb("POST");

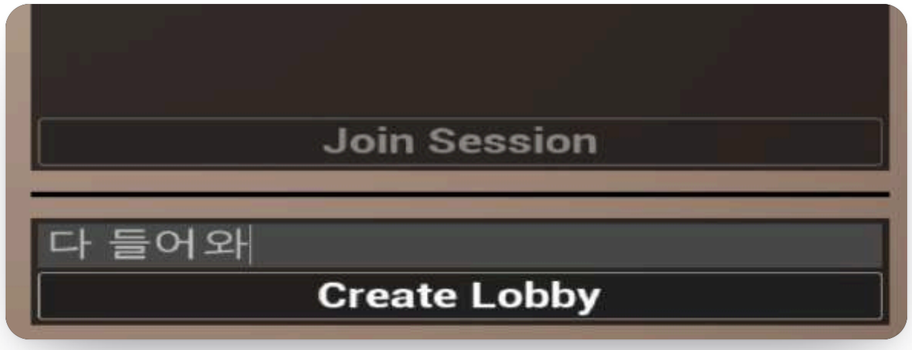
    Request->SetHeader( HeaderName TEXT("Content-Type"),  HeaderValue TEXT("application/json"));

    TSharedPtr<FJsonObject> JsonObject = MakeShareable(new FJsonObject);
    JsonObject->SetStringField(UCNetStatics::GetSessionNameKey().ToString(), NewSessionName.ToString());
    JsonObject->SetStringField(UCNetStatics::GetSessionSearchIdKey().ToString(), SessioinSearchId.ToString());

    FString RequestBoby;
    TSharedRef<TJsonWriter<>> Writer = TJsonWriterFactory<>::Create(&RequestBoby);
    FJsonSerializer::Serialize( Object JsonObject.ToSharedRef(), Writer);

    Request->SetContentAsString(RequestBoby);
    Request->OnProcessRequestComplete().BindUObject(this, &UCGameInstance::SessionCreationRequestCompleted,  [&] SessioinSearchId);

    if (!Request->ProcessRequest())
    {
        UE_LOG(LogTemp, Warning, TEXT("Sesison Creation Request Failed Right Away!"))
    }
}
```



01 세션 생성 요청

Create Lobby 클릭 시 코디네이터 서버의 /Sessions로 POST 요청

02 세션 정보 전달

세션 이름과 검색 ID를 JSON 본문으로 직렬화

03 로비 생성

요청 완료 후 SessionCreationRequestCompleted 콜백에서 로비 생성

코디네이터 서버

클라이언트의 세션 생성 요청을 받아 로컬에서 언리얼 서버 인스턴스를 실행하는 Flask 서버

주요 구현 내용

01 /Sessions 엔드포인트

Flask로 세션 생성 API 구성

02 세션 정보 수신

POST(JSON) 요청에서 세션 이름, 검색 ID 추출

03 언리얼 서버 실행

subprocess.Popen으로 UnrealEditor.exe -server 실행

04 포트 자동 증가

nextAvailablePort 증가로 포트 중복 방지

05 포트 반환

실행된 서버의 포트를 JSON으로 응답

정리 Flask REST API로 언리얼 서버 프로세스를 제어하고, 포트 자동 할당으로 세션 관리

Coordinator · Python Flask

```python
from flask import Flask, request, jsonify
import subprocess
from consts import SESSION_NAME_KEY, SESSION_SEARCH_ID_KEY, PORT_KEY
import re

app=Flask(__name__)

nextAvailablePort = 7777

def CreateSeverLocalTest(sessionName, sessionSearchId):
    global nextAvailablePort
    subprocess.Popen([
        "D:/UE5.5.4_Git/Engine/Binaries/Win64/UnrealEditor.exe",
        "D:/Unreal Projects/Crunch-master/Crunch.uproject" ,
        "-server",
        "-log",
        '-epicapp="ServerClient"',
        f'-SESSION_NAME="{sessionName}"',
        f'-SESSION_SEARCH_ID="{sessionSearchId}"',
        f'-PORT={nextAvailablePort}'
    ])

    usedPort = nextAvailablePort
    nextAvailablePort += 1
    return usedPort


@app.route('/Sessions', methods=['POST'])
def CreateSever():
    print(dict(request.headers))

    sessionName = request.get_json().get(SESSION_NAME_KEY)
    sessionSearchId = request.get_json().get(SESSION_SEARCH_ID_KEY)

    port = CreateSeverLocalTest(sessionName, sessionSearchId)
    return jsonify({"status": "success", PORT_KEY: port}), 200

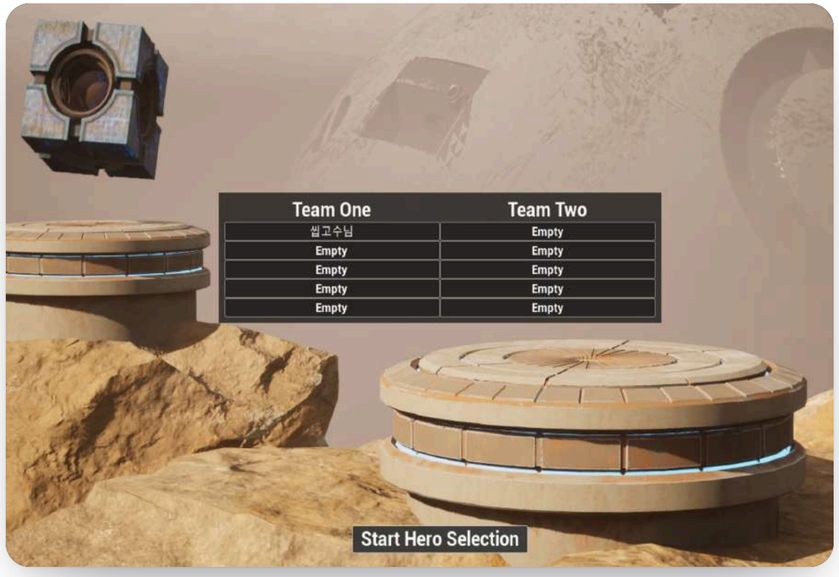
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=80)
```

세션 참여

호스트가 로비 생성 시 코디네이터 서버가 서버 실행 후 EOS 세션 생성

호스트 로비 생성 후

서버 준비 완료 시 호스트가 EOS 세션 정보로 먼저 접속



방 목록

게스트는 목록 갱신 후 세션 선택



해당 방 참여 후

Join Session 클릭 시 세션 접속, 호스트와 같은 로비 입장



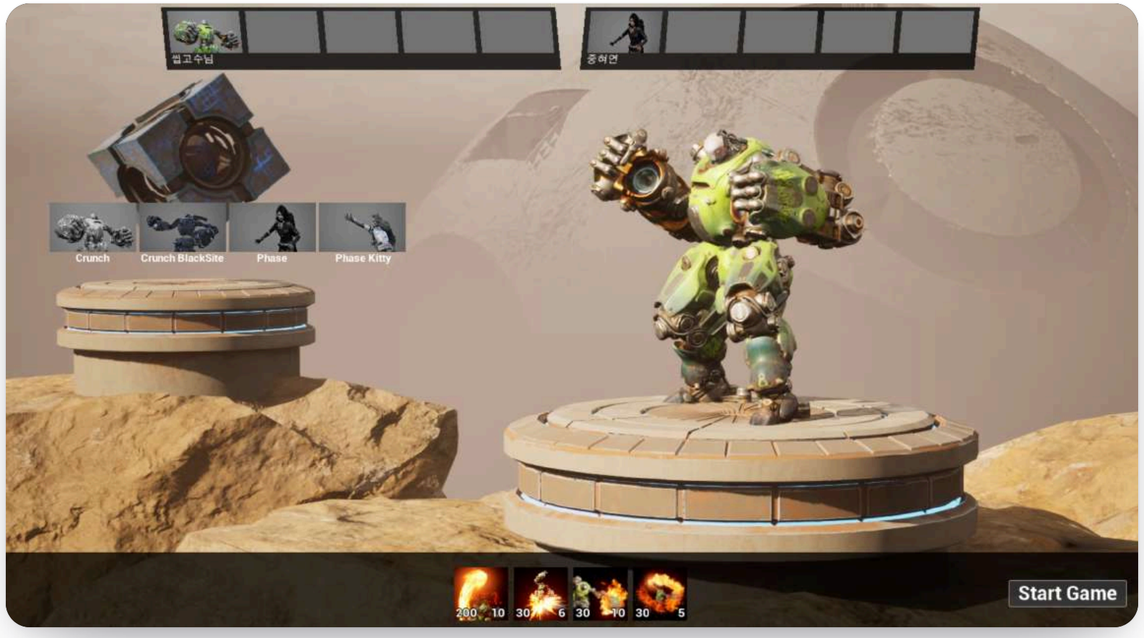
다음 단계 호스트가 **Start Hero Selection** 실행 시 영웅 선택 단계로 진행

에셋 매니저 · PrimaryDataAsset

AssetManager로 캐릭터 선택창의 캐릭터 데이터 로드

로비 캐릭터 선택창

에셋 매니저로 불러온 캐릭터 데이터로 선택창 구성

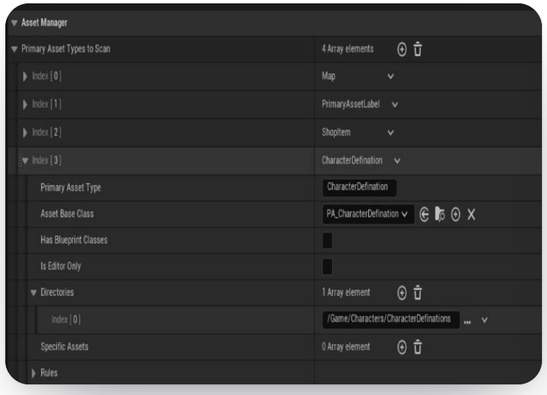


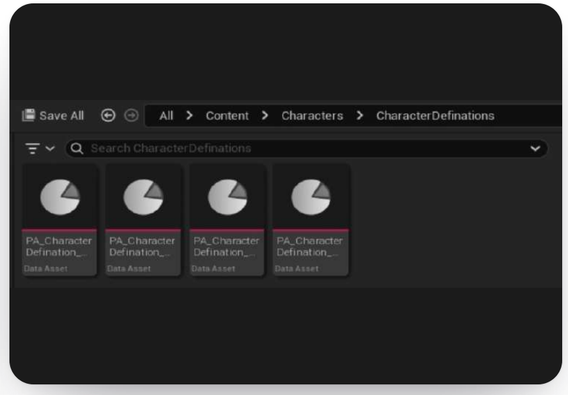
LoadObject() 대신 Primary Asset 시스템

디렉터리 단위로 에셋을 불러오고 에셋 매니저에서 일괄 관리

타입 · 경로 설정, 로드 코드

Primary Asset 타입과 디렉터리 지정 후 일괄 로드





UCAssetManager::GetLoadedCharacterDefinations

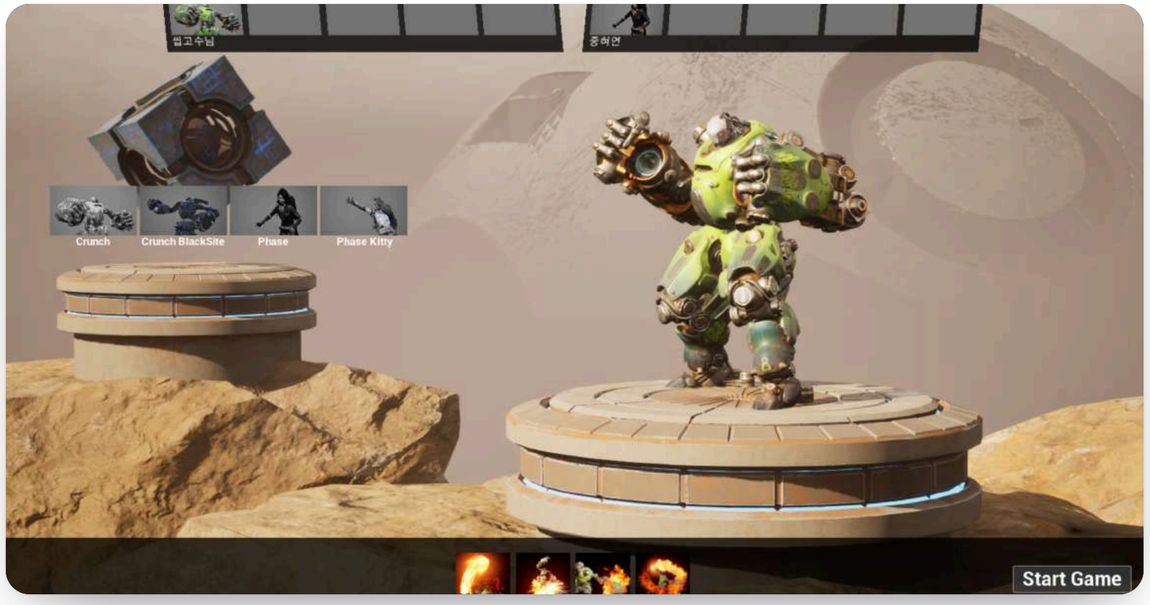
```
bool UCAssetManager::GetLoadedCharacterDefinations(TArray<UPA_CharacterDefination*>& LoadedCharacterDefinations) const
{
    TArray<UObject*> LoadedObjects;
    bool bLoaded = GetPrimaryAssetObjectList(UPA_CharacterDefination::GetCharacterDefinationAssetType(), LoadedObjects);
    if (bLoaded)
    {
        for (UObject* LoadedObject : LoadedObjects)
        {
            LoadedCharacterDefinations.Add( Item: Cast<UPA_CharacterDefination>(LoadedObject));
        }
    }

    return bLoaded;
}
```

02 · LOGIN & LOBBY

로비 캐릭터 선택

선택 정보는 PlayerState에 저장, GameState에서 처리 후 모든 클라이언트 UI 동기화



Crunch 선택 화면

상단 팀 슬롯에 플레이어별 선택 캐릭터 표시



Phase 선택 화면

하단 툴팁으로 캐릭터 스킬 정보 확인

01

캐릭터 선택

선택창에서 원하는 캐릭터 선택

02

중복 선택 방지

다른 플레이어가 선택한 캐릭터는 선택 불가

03

UI 동기화

PlayerState 저장 → GameState 처리 → 전체 클라이언트 전달

04

게임 시작

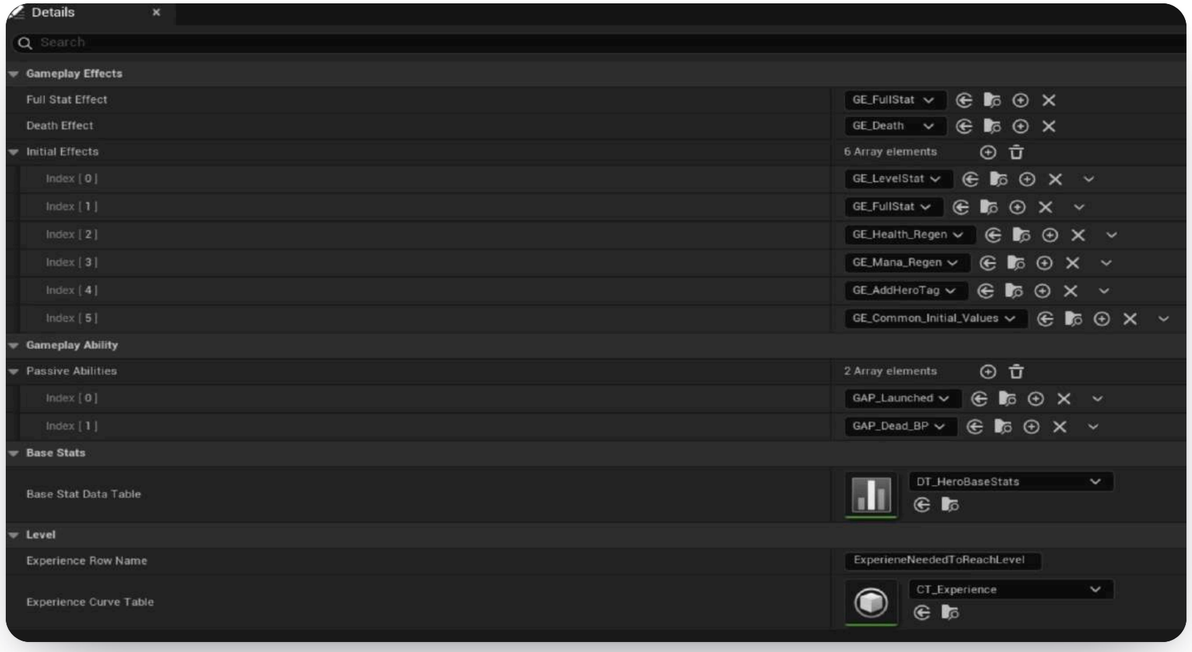
호스트의 Start Game으로 선택 확정 후 게임 시작

캐릭터 기본 스탯

GAS 기반, 기본 스탯(Attribute)을 데이터 중심으로 서버에서 초기화

캐릭터 GAS 설정

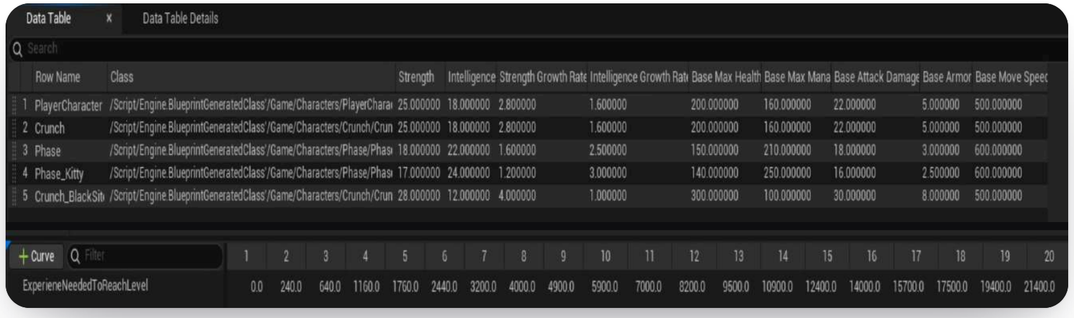
초기 이펙트, 패시브 어빌리티, 스탯 테이블, 경험치 커브 지정



데이터 에셋 · 테이블로 통합 관리 GAS 연동으로 초기 Attribute 설정을 일관되게 처리

데이터 테이블 · 커브 테이블

클래스별 기본 능력치, 레벨별 경험치 요구량



01 GameplayEffect

GE로 AttributeSet 값 설정 및 태그 부여

02 BaseStat 데이터 테이블

클래스별 체력, 공격력, 방어력 등 기본 능력치 초기화

03 경험치 커브 테이블

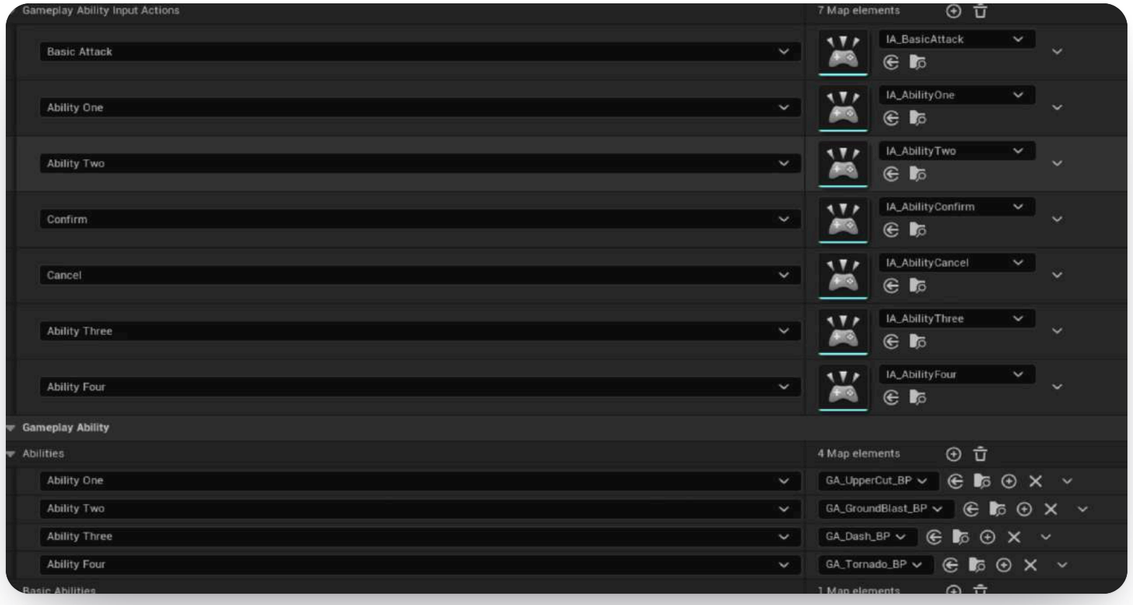
CT_Experience로 레벨별 경험치 요구량 자동 세팅

캐릭터 어빌리티

Enum – Input – Ability 매핑 구조로 캐릭터별 고유 어빌리티 관리

어빌리티 입력 매핑

입력 액션과 어빌리티 클래스를 캐릭터별로 매핑



어빌리티 부여 코드

초기화 시점에 GiveAbility()로 어빌리티 부여

UCAbilitySystemComponent::GiveInitialAbilities

```
void UCAbilitySystemComponent::GiveInitialAbilities()
{
    if (!GetOwner() || !GetOwner()->HasAuthority())
        return;

    for (const TPair<ECAbilityInputID,TSubclassOf<UGameplayAbility>>& AbilityPair : Abilities)
    {
        GiveAbility(FGameplayAbilitySpec(AbilityPair.Value, InLevel: 0, InInputID: (int32)AbilityPair.Key, InSourceObject: nullptr));
    }

    for (const TPair<ECAbilityInputID,TSubclassOf<UGameplayAbility>>& AbilityPair : BasicAbilities)
    {
        GiveAbility(FGameplayAbilitySpec(AbilityPair.Value, InLevel: 1, InInputID: (int32)AbilityPair.Key, InSourceObject: nullptr));
    }

    if (!AbilitySystemGenerics)
        return;

    for (const TSubclassOf<UGameplayAbility>& PassiveAbility : AbilitySystemGenerics->GetPassiveAbilities())
    {
        GiveAbility(FGameplayAbilitySpec(PassiveAbility, InLevel: 1, InInputID: -1, InSourceObject: nullptr));
    }
}
```

01

초기화 시 부여

게임 시작 시 설정된 어빌리티를 GiveAbility()로 부여

02

입력 ID로 실행

입력 발생 시 AbilityLocalInputPressed((int32)InputID)로 매핑된 어빌리티 실행

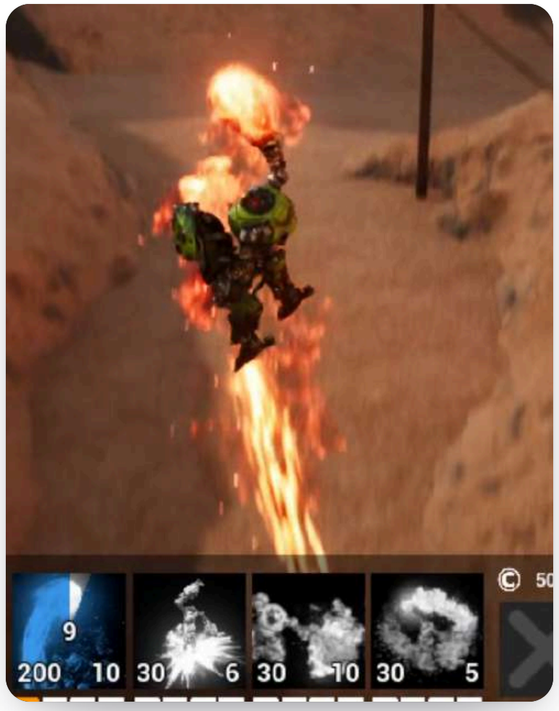
03

확장성

새 스킬이나 입력 방식이 추가돼도 매핑만 추가하면 되는 구조

Crunch

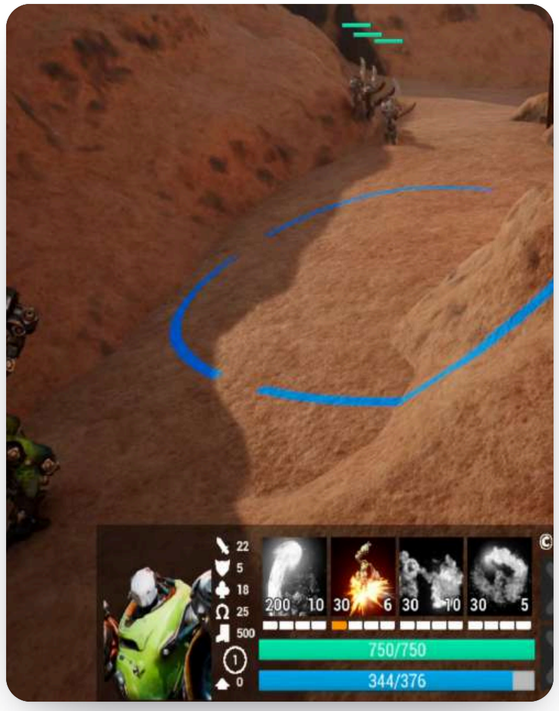
근접 캐릭터 · 스킬 4종



01 어퍼컷

적을 위로 띄워 올리며 데미지

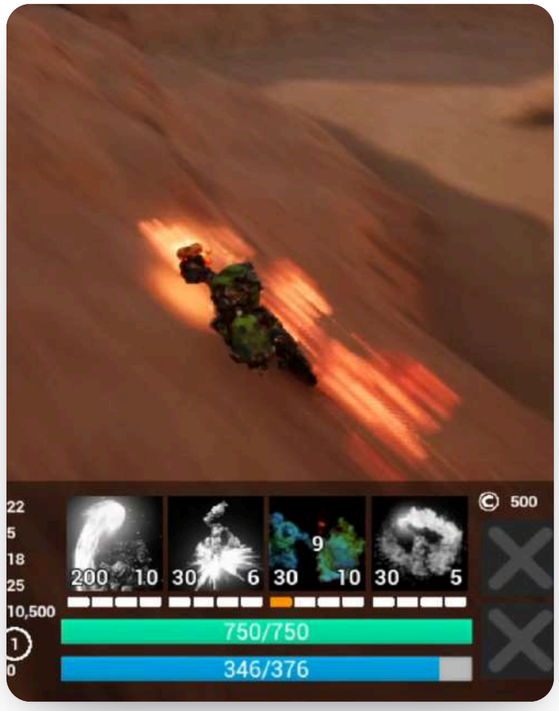
Ability One · GA_UpperCut_BP



02 그라운드 블래스트

지면 폭발로 범위 내 적에게 데미지

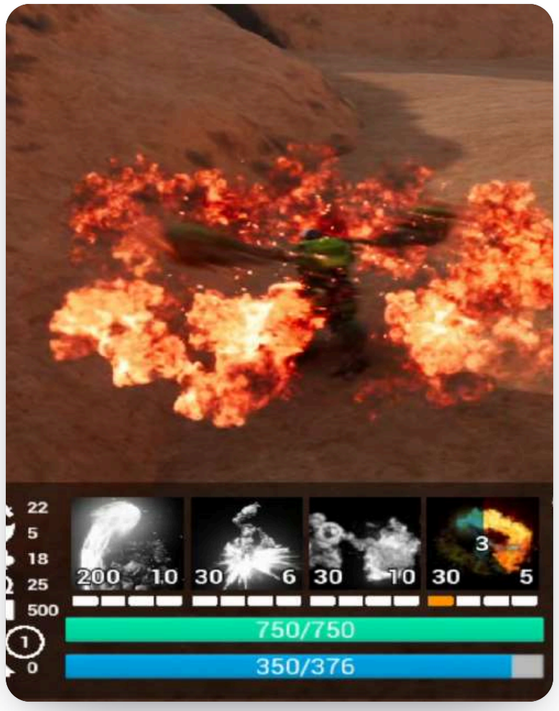
Ability Two · GA_GroundBlast_BP



03 대시

주먹을 뻗으며 돌진, 부딪힌 적에게 데미지

Ability Three · GA_Dash_BP



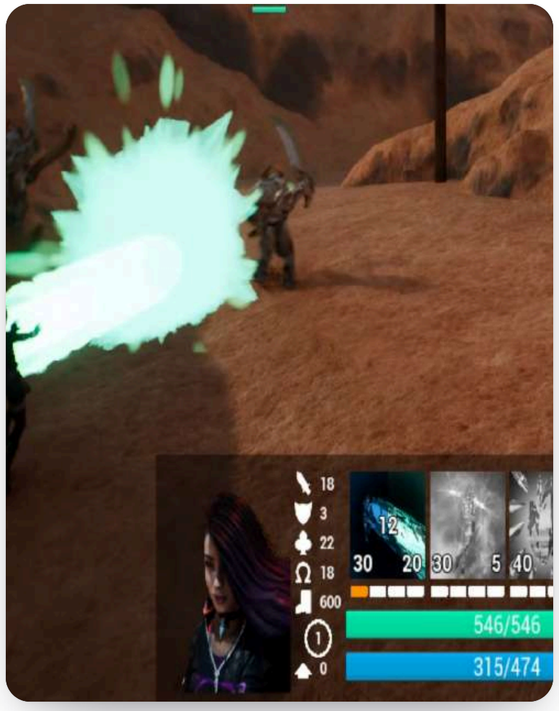
04 토네이도

2초간 주먹을 회전시키며 닿는 적을 날려 버림

Ability Four · GA_Tornado_BP

Phase

마나 기반 캐릭터 · 스킬 4종



01 밀어내기

레벨 비례 마나 소모, 닿는 적에게 데미지 후 넉백

Ability One



02 순간이동

지정한 위치로 순간이동

Ability Two



03 얼음 소환

범위에 얼음 소환, 데미지와 함께 이동 속도 감소

Ability Three



04 블랙홀

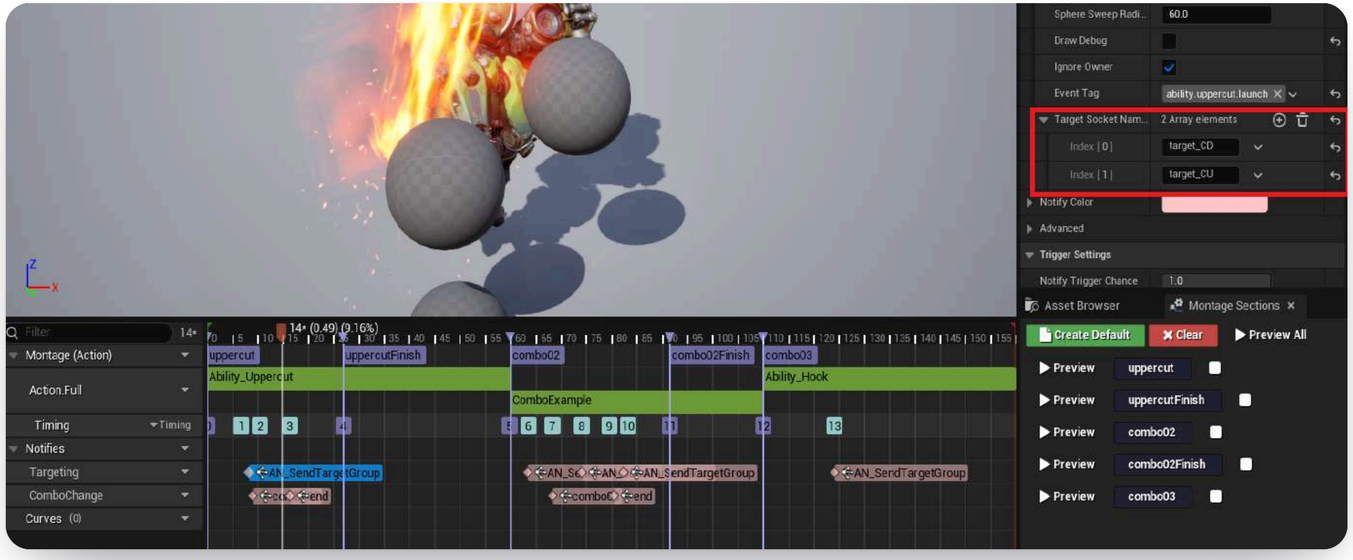
주변 적을 끌어당긴 뒤 4초 후 폭발

Ability Four

데미지 처리 문제 해결

충돌체 대신 소켓 간 Sweep으로 근접 공격 충돌 판정

문제 애니메이션은 시간 기준, 충돌 검사는 프레임 기준으로 동작. **저프레임이나 빠른 애니메이션에서 충돌이 스킵되어 데미지 누락**



Sweep AnimNotify

```cpp
for (int i = 1; i < TargetSocketNames.Num(); ++i)
{
    FVector StartLoc = MeshComp->GetSocketLocation(TargetSocketNames[i-1]);
    FVector EndLoc = MeshComp->GetSocketLocation(TargetSocketNames[i]);

    TArray<TEnumAsByte<EObjectTypeQuery>> ObjectTypes;
    ObjectTypes.Add( Item: UEngineTypes::ConvertToObjectType( CollisionChannel: ECC_Pawn));
    TArray<FHitResult> HitResults;

    TArray<AActor*> ActorsToIgnore;
    if (bIgnoreOwner)
    {
        ActorsToIgnore.Add(OwnerActor);
    }

    EDrawDebugTrace::Type DrawDebugTrace = bDrawDebug ? EDrawDebugTrace::ForDuration : EDrawDebugTrace::None;

    UKismetSystemLibrary::SphereTraceMultiForObjects(MeshComp, StartLoc, EndLoc, SphereSweepRadius,
        ObjectTypes, bTraceComplex: false, ActorsToIgnore, DrawDebugTrace, [&] HitResults, bIgnoreSelf: false);
```

01
소켓 경로 순회
배열에 지정한 소켓 위치를 순서대로 순회

02
Sphere Trace
소켓 간 경로를 한 프레임 안에서 스피어 트레이스로 검사

03
타겟 처리
트레이스 결과로 충돌 대상 감지 및 처리

결과

애니메이션 속도, 기기 성능과 무관하게 **일관된 충돌 판정**

04 · INVENTORY

인벤토리 컴포넌트

아이템 보유 · 구매 · 판매 · 사용 로직 담당. 데이터는 서버에서 관리, 클라이언트는 UI 표시와 요청만 수행

인게임 인벤토리

스탯, 스킬, 골드, 아이템 슬롯, 체력 · 마나 표시



서버 기반 관리 구조

01 서버 처리

구매, 판매, 추가, 삭제 등 인벤토리 로직 전부 서버에서 처리

02 RPC 요청 · 검증

클라이언트는 RPC로 요청만 전송, 서버가 유효성 검증 후 반영

03 클라이언트 수정 불가

보안 및 동기화 문제 방지

서버 RPC

Server, Reliable, WithValidation RPC로 구매 · 사용 · 판매 요청 처리

InventoryComponent · Server RPC

```
/*****************************************************/
/*                    Server                         */
/*****************************************************/
UFUNCTION(Server, Reliable, WithValidation)
void Server_Purchase(const UPA_ShopItem* ItemToPurchase);

UFUNCTION(Server, Reliable, WithValidation)
void Server_ActivateItem(FInventoryItemHandle ItemHandle);

UFUNCTION(Server, Reliable, WithValidation)
void Server_SellItem(FInventoryItemHandle ItemHandle);
void GrantItem(const UPA_ShopItem* NewItem);
void ConsumeItem(UInventoryItem* Item);
void RemoveItem(UInventoryItem* Item);
bool TryItemCombination(const UPA_ShopItem* NewItem);
```

네트워크 최적화

TArray 대신 **TMap<FInventoryItemHandle, UInventoryItem*>** 사용. 고유 핸들로 아이템 식별, 데이터에는 수량 · 옵션 · 강화 정보 저장

InventoryItemHandle

구조체 전체 대신 고유 ID만 주고받아 네트워크 부담 감소

FInventoryItemHandle

고유 ID(정수) 하나만 가진 구조체. 네트워크로는 ID만 전송하고, 실제 데이터는 서버와 클라이언트가 각자 로컬에 보유

FInventoryItemHandle

```
USTRUCT()
struct FInventoryItemHandle
{
    GENERATED_BODY()
public:
    FInventoryItemHandle();
    static FInventoryItemHandle InvalidHandle();
    static FInventoryItemHandle CreateHandle();

    bool IsValid() const;
    uint32 GetHandleId() const { return HandleId; }
private:
    explicit FInventoryItemHandle(uint32 Id);

    UPROPERTY()
    uint32 HandleId;

    static uint32 GenerateNextId();
    static uint32 GetInvalidId();
};

bool operator==(const FInventoryItemHandle& Lhs, const FInventoryItemHandle& Rhs);
uint32 GetTypeHash(const FInventoryItemHandle& Key);
```

문제

인벤토리를 배열이나 구조체 그대로 Replicate하면 **데이터 크기만큼 네트워크 부담 증가**

아이템 추가 흐름

구매 요청 시 서버 검증 후 아이템 전달. 클라이언트는 Handle과 아이템을 담은 Map을 로컬에서 관리

Server · Add Item

```
UInventoryItem* InventoryItem = NewObject<UInventoryItem>();
FInventoryItemHandle NewHandle = FInventoryItemHandle::CreateHandle();
InventoryItem->InitItem(NewHandle, NewItem, OwnerAbilitySystemComponent);
InventoryMap.Add(NewHandle, InventoryItem);
OnItemAdded.Broadcast(InventoryItem);
UE_LOG(LogTemp, Warning, TEXT("Server Adding Shop Item: %s, with Id: %d"),
    *(InventoryItem->GetShopItem()->GetItemName().ToString()), NewHandle.GetHandleId());
FGameplayAbilitySpecHandle GrantedAbilitySpecHandle = InventoryItem->GetGrantedAbilitySpecHandle();
Client_ItemAdded(NewHandle, NewItem, GrantedAbilitySpecHandle);
```

결과 이후 서버와는 **Handle 값만** 주고받아 네트워크 최적화

미니언 스폰 · 오브젝트 풀링

미리 생성한 미니언을 재사용해 스폰 비용 절감

문제 SpawnActor로 매번 생성 · 제거 시 메모리 할당/해제와 액터 초기화 반복. **대량 스폰 시 CPU, GC 비용 증가**

그룹 단위 스폰 · 활성화

한 그룹 단위로 풀에서 미니언을 꺼내 활성화

AMinionBarrack::SpawnNewGroup

```cpp
void AMinionBarrack::SpawnNewGroup()
{
    int i = MinionPerGroup;

    while (i > 0)
    {
        FTransform SpawnTransfrom = GetActorTransform();
        if (const APlayerStart* NextSpawnSpot = GetNextSpawnSpot())
        {
            SpawnTransfrom = NextSpawnSpot->GetActorTransform();
        }

        AMinion* NextAvaliableMinon = GetNextAvaliableMinion();
        if (!NextAvaliableMinon)
            break;

        NextAvaliableMinon->SetActorTransform(SpawnTransfrom);
        NextAvaliableMinon->Activate();
        --i;
    }

    SpawnNewMinions(i);
}
```

01 스폰 위치

GetNextSpawnSpot()으로 다음 스폰 위치 조회

02 풀에서 꺼내기

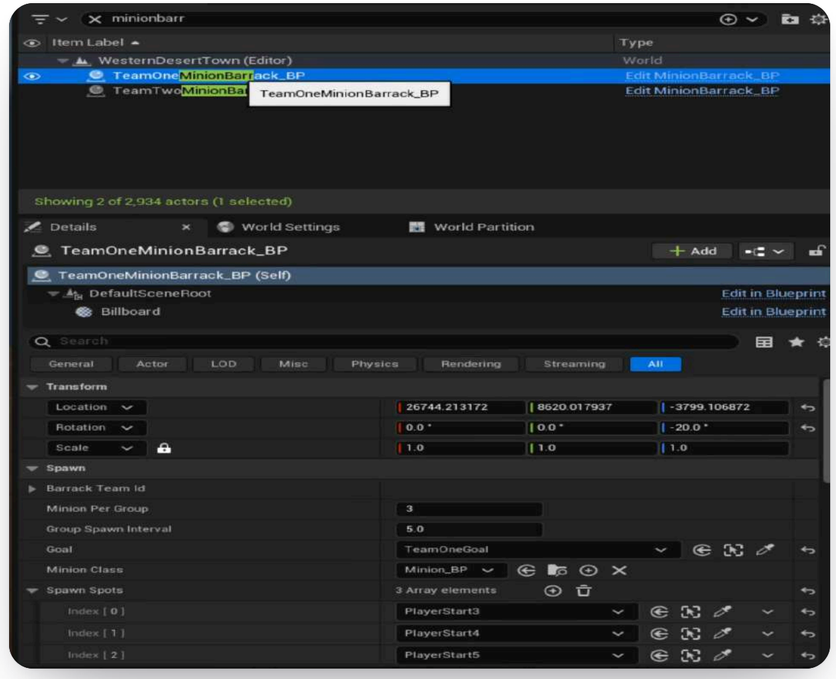
GetNextAvaliableMinion()으로 사용 가능한 미니언 조회. 사용 가능한 미니언은 Dead 태그 보유

03 위치 지정 · 활성화

위치 지정 후 Activate(). Dead 태그를 제거해 다음 스폰에서 제외

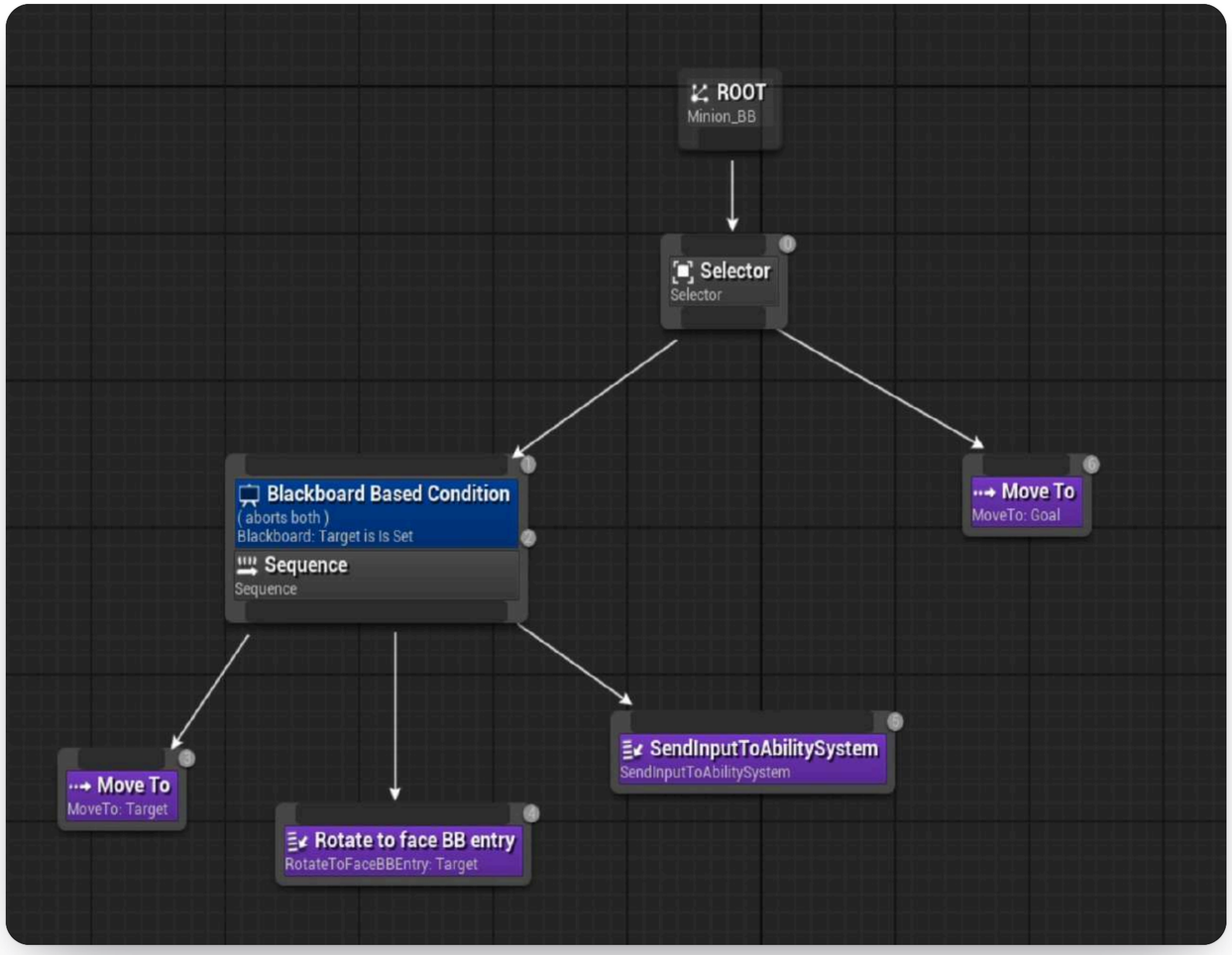
레벨 배치 · 설정

스폰 위치, 목표 위치, 팀 ID 지정 후 시작 시 스폰



Behavior Tree

타겟 유무에 따라 공격 또는 목표 지점으로 이동



SendInputToAbilitySystem

BT 태스크에서 AbilitySystemComponent로 입력 ID를 전달해 어빌리티 실행

UBTTask_SendInputToAbilitySystem::ExecuteTask

```
EBTNodeResult::Type UBTTask_SendInputToAbilitySystem::ExecuteTask(UBehaviorTreeComponent& OwnerComp, uint8* NodeMemory)
{
    AAIController* OwnerAIC = OwnerComp.GetAIOwner();
    if (OwnerAIC)
    {
        UAbilitySystemComponent* OwnerASC = UAbilitySystemBlueprintLibrary::GetAbilitySystemComponent( Actor: OwnerAIC->GetPawn());
        if (OwnerASC)
        {
            OwnerASC->PressInputID((int32)InputID);
            return EBTNodeResult::Succeeded;
        }
    }

    return EBTNodeResult::Failed;
}
```

타겟 있음

적 방향으로 회전 후 **어빌리티로 기본 공격**

타겟 없음

목표 지점(Goal)으로 계속 이동

06 · CARGO

화물 밀기

점령 인원 차이에 비례해 목표 이동, 목표 지점 도착 시 해당 팀 승리



AStormCore::InfluencerLeftRange

```
void AStormCore::InfluencerLeftRange(UPrimitiveComponent* OverlappedComponent, AActor* OtherActor, UPrimitiveComponent* OtherComp, int32 OtherBodyIndex)
{
    IGenericTeamAgentInterface* OtherTeamInterface = Cast<IGenericTeamAgentInterface>(OtherActor);
    if (OtherTeamInterface)
    {
        if (OtherTeamInterface->GetGenericTeamId().GetId() == 0)
        {
            TeamOneInfluncerCount--;
            if(TeamOneInfluncerCount< 0)
            {
                TeamOneInfluncerCount = 0;
            }
        }
        else if (OtherTeamInterface->GetGenericTeamId().GetId() == 1)
        {
            TeamTwoInfluncerCount--;
            if(TeamTwoInfluncerCount< 0)
            {
                TeamTwoInfluncerCount = 0;
            }
        }
        UpdateTeamWeight();
    }
}
```

AStormCore::UpdateTeamWeight

```
void AStormCore::UpdateTeamWeight()
{
    OnTeamInfluenceCountUpdated.Broadcast(TeamOneInfluncerCount, TeamTwoInfluncerCount);
    if (TeamOneInfluncerCount == TeamTwoInfluncerCount)
    {
        TeamWeight = 0.f;
    }
    else
    {
        float TeamOffset = TeamOneInfluncerCount - TeamTwoInfluncerCount;
        float TeamTotal = TeamOneInfluncerCount + TeamTwoInfluncerCount;

        TeamWeight = TeamOffset / TeamTotal;
    }

    UpdateGoal();
}
```

01

점령 인원 집계

화물 범위 안의 팀별 인원 수 갱신

02

팀 가중치 계산

(팀1 − 팀2) / 전체 인원으로 이동 방향과 속도 결정

03

승리 조건

목표 지점 도착 시 해당 팀 승리

CONTACT

Thank you.

이름	안중현
이메일	sie08357@naver.com
전화	010-2070-2986

GitHub ↗ 플레이 영상 ↗ 블로그 ↗

